How to configure and upload data to git hub

Uploading data or a project to GitHub is a fundamental skill for any developer. The most robust way to do this is by using the Git Command Line Interface (CLI).

Here is a step-by-step guide to configuring Git and uploading your local files to a GitHub repository. Aidan Nguyen - Medium

1. Prerequisites

Install Git: If you haven't already, download and install Git from git-scm.com.

GitHub Account: Ensure you have an account created on GitHub.

2. Configure Git (One-Time Setup)

Before uploading files, you need to tell Git who you are so your commits are properly attributed to your account. Open your terminal (or Git Bash on Windows) and run these commands:

Bash

```
# Set your username
git config --global user.name "Your Name"

# Set your email address (use the one associated with your GitHub account)
git config --global user.email "your.email@example.com"
```

3. Create a New Repository on GitHub

1. Log in to GitHub.

2. In the top right corner, click the **+** icon and select **New repository**.
   Aidan Nguyen - Medium
3. Give your repository a short, memorable name (e.g., `my-project`).
4. Choose whether to make it **Public** or **Private**.
5. Leave the "Initialize this repository with..." options unchecked for now, as you will be uploading existing files.
6. Click **Create repository**. Aidan Nguyen - Medium

4. Initialize and Upload Your Local Project

Open your terminal and navigate to the folder on your computer where your project files are located.

Step A: Navigate to your project directory

Bash

```
cd path/to/your/project-folder
```

Step B: Initialize Git

This creates a hidden `.git` folder that tracks your project's history.

Bash

```
git init
```

Step C: Add your files to the staging area

The `.` tells Git to stage all the files in your current directory.

Bash

```
git add .
```

Step D: Commit your files

A commit takes a snapshot of your files. The `-m` flag allows you to add a descriptive message.

Bash

```
git commit -m "Initial commit"
```

Step E: Set the main branch (Optional but recommended)

GitHub now uses `main` as the default branch name instead of `master`.

Bash

```
git branch -M main
```

Step F: Link to your GitHub repository

Go to the repository page you just created on GitHub and copy the HTTPS URL. Connect your local directory to this remote repository:

Bash

```
# Replace the URL below with your actual repository URL
git remote add origin https://github.com/your-username/your-repository-name.gi
```

Step G: Push your files to GitHub

Upload your committed files to the remote repository.

Bash

```
git push -u origin main
```

Note: You may be prompted to authenticate with your GitHub credentials during this step.

Alternative: The Web UI Method

If you only have a few files to upload and don't want to use the command line, you can use the web interface:

1. Create a repository on GitHub (check the box to "Add a README file" so the repository initializes).
2. Open the repository, click the **Add file** dropdown, and select **Upload files.** GitHub
3. Drag and drop your files into the browser, add a commit message at the bottom, and click **Commit changes.** Data Science Modules

```
PS C:\NS\AI_Automation> git push -u origin main
To https://github.com/swamy7github/Fyers_AI_Client.git
 ! [rejected]        main -> main (non-fast-forward)
error: failed to push some refs to
```

This is a very common Git error. It happens because your repository on GitHub contains files that do not exist on your local computer.

This usually occurs if you checked the box to **"Add a README file"**, `.gitignore`, or a License when you first created the repository on GitHub. Git blocks your push to prevent you from accidentally overwriting those remote files.

Here are the two ways to fix it, depending on what you want to keep.

Option 1: Merge the GitHub files with your local files (Recommended)

If you want to keep the files that are currently on GitHub (like a README) and combine them with your local project, you need to pull those files down first.

Since your local project and the GitHub project were started separately, you need to tell Git to allow them to merge:

1. Pull the remote files:

Bash

```
git pull origin main --allow-unrelated-histories
```

*(Note: This might open a terminal text editor like Vim asking for a commit message. If your screen suddenly changes, just type `:wq` and press **Enter** to save the default message and exit).*

2. Push everything back up to GitHub:

Bash

```
git push -u origin main
```

Option 2: Force Push (Quickest, but overwrites GitHub)

If the GitHub repository is brand new and you don't care about any files currently sitting on it (e.g., it's just an empty README), you can force Git to overwrite the remote repository with your exact local copy.

Run this command:

Bash

```
git push -f origin main
```

Warning: Only use `-f` (force) if you are absolutely sure you don't need whatever is currently on the GitHub repository.

Need help ignoring unnecessary files?

↪ Yes

How do I create a .gitignore file so I don't accidentally upload virtual environments or compiled files?

Creating a `.gitignore` file ensures Git ignores specific files and folders, keeping your repository clean, saving upload time, and protecting sensitive data.

1. Create the File

Since you are working in PowerShell, you can create the file directly in your project folder by running:

PowerShell

```
New-Item .gitignore -ItemType File
```

Alternatively, you can simply create a new file in your code editor (like VS Code) and name it exactly `.gitignore` (make sure it starts with the dot and has no `.txt` extension).

2. Add Ignore Rules

Open the `.gitignore` file in your editor and add the patterns you want Git to ignore.

Based on typical automation and client-side projects, here is a standard template that blocks virtual environments, compiled files, and sensitive keys:

Plaintext

```
# Virtual Environments
venv/
env/
.venv/

# Compiled Files & Caches
__pycache__/
*.pyc
*.class
*.exe
*.dll
build/
dist/

# Environment Variables & Secrets (CRITICAL)
.env
*.pem
keys.json

# IDE and OS junk files
.vscode/
.idea/
.DS_Store
Thumbs.db
```

> **Syntax tip:** *A trailing slash (`/`) tells Git to ignore an entire folder. An asterisk (`*`) acts as a wildcard (e.g., `*.pyc` ignores any file ending in that extension).*

3. Save and Commit

Once you have saved your `.gitignore` file, you need to track it in your repository just like any regular code file:

Bash

```
git add .gitignore
git commit -m "Add .gitignore to exclude virtual environments and compiled fil
git push
```

Important: What if the files are already tracked?

If you previously ran `git add .` or committed your virtual environment *before* making the `.gitignore` file, Git will continue to track those files. A `.gitignore` only prevents files from being added; it does not automatically remove files that were already committed.

To clear Git's cache and force it to respect your new `.gitignore`, run these three commands in order:

Bash

```
# 1. Untrack all files (this removes them from Git's memory, but keeps your ac
git rm -r --cached .

# 2. Re-add everything (this time, Git will read the .gitignore and skip the i
git add .

# 3. Commit the clean slate
git commit -m "Apply .gitignore and clear previously tracked files"
```

Want to learn how to securely manage API keys?

Command History

Id	CommandLine
1	gitt
2	git
3	cd ..\..\NS\AI_Automation\
4	git config --global user.name
5	git config --global user.email
6	git init
7	git add .
8	git commit -m "Initial commit"
9	git branch -M main
10	git status
11	git add.
12	git add .
13	git status
14	git commit --amne d
15	git commit --amned
16	git commit --amend
17	git remote add origin https://github.com/swamy7github/Fyers_AI_Client.git
18	git push -u origin main
19	git pull
20	git pull Fyers_AI_Client main
21	git branch
22	git beanch L1
23	git branch L1
24	git branch
25	git pull main
26	git push -u origin main
27	git pull
28	git branch --set-upstream-to=origin/main main
29	git push -u origin main
30	git branch
31	git diff
32	git status
33	git pull origin main --allow-unrelated-histories
34	git push -u origin main

branch 'main' set up to track 'origin/main' — How To

This message appears when a local branch is linked to a remote branch so that `git push` and `git pull` know where to sync by default.

Option 1: Automatically, while pushing (most common)

```
git push -u origin main
```

The `-u` flag (short for `--set-upstream`) pushes your `main` branch to `origin` and sets up the tracking link in the same step. After this, you can just run `git push` or `git pull` without specifying `origin main` every time.

Option 2: Set tracking without pushing

```
git branch --set-upstream-to=origin/main main
```

Use this when the remote branch already exists and you just want to link to it — useful when the branches exist on both sides but aren't linked yet (e.g., after cloning differently or fixing a mismatched setup).

Option 3: While creating a new branch

```
git checkout -b main origin/main
```

This creates `main` locally from `origin/main` and sets tracking automatically.

How to check it worked

```
git branch -vv
```

This shows each local branch and which remote branch it's tracking, like:

```
* main abc1234 [origin/main] Initial commit
```

Quick fix: if you just want to link an already-existing repo, `git branch --set-upstream-to=origin/main main` (Option 2) is the quickest route.